

Your extraction pipeline validates outputs against JSON schemas, but you need to implement human review given limited reviewer capacity (they can handle approximately 5% of total extraction volume). What's the most effective basis for selecting which extractions to route for human review?

- ☒ **A** Route extractions where the model indicates low confidence or where source documents contain ambiguous or contradictory information.
- ☐ **B** Randomly sample 5% of extractions for review.
- ☐ **C** Route extractions for review only when downstream systems report data quality issues or processing failures.
- ☐ **D** Route extractions containing specific high-priority entity types (e.g., financial figures, dates) for human review, regardless of extraction confidence.

Testing reveals that when source documents are missing certain specifications, the model fabricates plausible-sounding values to satisfy your schema's required fields. For example, a document mentioning only dimensions receives a fabricated "weight: 2.3 kg" in the extraction output. What schema design change most effectively addresses this hallucination behavior?

- ☐ **A** Add explicit instructions to the prompt stating "only extract information explicitly stated in the document; use placeholder text for missing values"
- ☐ **B** Implement semantic validation that verifies each extracted value appears in or can be inferred from the source document text
- ☒ **C** Change fields that may not exist in source documents from required to optional, allowing the model to omit them
- ☐ **D** Add a "confidence" field alongside each specification where the model self-reports its certainty, then filter out low-confidence extractions

Your extraction system implements automatic retries when validation fails. On each retry, the specific validation error is appended to the prompt. This retry-with-error-feedback approach resolves most failures within 2-3 attempts. For which failure pattern would additional retries be LEAST effective?

- ☐ **A** The model extracts citation counts as locale-formatted strings ("1,234") when the schema requires integers
- ☒ **B** The model extracts "et al." for co-authors when the full list exists only in an external document not in the input
- ☐ **C** The model extracts dates as ISO 8601 datetime strings ("2023-03-15T00:00:00Z") when the schema requires only the date portion (YYYY-MM-DD)
- ☐ **D** The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings

Your system extracts event metadata (date, location, organizer, attendee_count) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article—for example, outputting "500" for attendee_count when the source contains no attendance information. What's the most effective way to reduce these false extractions?

- ☒ **A** Add prompt instructions to return null for any field where information is not directly stated in the source.
- ☐ **B** Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.
- ☐ **C** Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.
- ☐ **D** Add a post-processing step using a second LLM call to verify each extracted value exists in the source document.

Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies. What's the most effective approach to improve extraction accuracy for documents with amendments?

- ☐ **A** Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms.
- ☐ **B** Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review.
- ☐ **C** Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step.
- ☒ **D** Redesign the schema so amended fields capture multiple values, each with source location and effective date.

Your extraction uses tool use with a JSON schema where `property_type` is defined as an enum: `['house', 'apartment', 'condo', 'townhouse']`. After deployment, 8% of extractions fail schema validation. Investigation reveals listings mention many uncommon property types—"studio", "loft", "duplex", "mobile home", "tiny house", "converted warehouse"—and new types continue appearing regularly. What's the most effective long-term solution?

- ☐ **A** Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.
- ☒ **B** Change `property_type` from an enum to a free-form string and implement a normalization step in post-processing.
- ☐ **C** Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.
- ☐ **D** Add an "other" value to your enum with a separate `property_type_detail` string field for specifics when "other" is selected.

The extraction pipeline receives documents of varying types—some are invoices, others are contracts, and some are receipts. You've defined separate extraction tools, each with its own JSON schema tailored to the document type. During testing, you observe that with `tool_choice: "auto"`, Claude sometimes returns conversational text instead of calling an extraction tool, causing downstream parsing failures. You need guaranteed structured output without knowing the document type in advance. What's the most effective approach?

- ☐ **A** Add a preliminary classification call, then make a second call with `tool_choice` forced to the identified extraction tool.
- ☐ **B** Set `tool_choice: "any"` with all extraction tools defined.
- ☐ **C** Keep `tool_choice: "auto"` with system prompt instructions requiring tool use.
- ☒ **D** Consolidate all document types into a single unified-schema extraction tool and force that tool.

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned." What's the most effective improvement?

- ☒ **A** Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.
- ☐ **B** Enrich the schema to `{skill: string, confidence: float, source_quote: string}[]` to capture extraction metadata.
- ☐ **C** Add post-extraction normalization that maps skills to a canonical taxonomy and deduplicates similar entries.
- ☐ **D** Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

Your pipeline uses a tool called `extract_metadata` with a JSON schema for paper details. You've also defined `lookup_citations` and `verify_doi` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls `lookup_citations` first, which fails because it needs the DOI that `extract_metadata` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- ☐ **A** Set `tool_choice` to `"auto"` and reorder the tool definitions so `extract_metadata` appears first in the tools array, since Claude prioritizes earlier-listed tools.
- ☒ **B** Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.
- ☐ **C** Set `tool_choice` to `"any"` so Claude must use a tool, combined with system prompt instructions prioritizing `extract_metadata`.
- ☐ **D** Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.

The system needs to extract candidate information (name, contact details, skills, work experience, education) from uploaded resumes. The extracted data must strictly conform to a predefined JSON schema, as missing required fields or incorrect data types will cause downstream validation failures. What is the most reliable approach to ensure Claude's output consistently matches the schema?

- ☐ **A** Make two separate API calls—first extracting information as text, then asking Claude to format that text as JSON.
- ☐ **B** Parse Claude's text response with regex patterns to extract JSON objects, using retry logic for malformed responses.
- ☒ **C** Define a tool with an input schema matching your required JSON structure and extract the data from Claude's tool_use response.
- ☐ **D** Include detailed JSON formatting instructions and a template example in the system prompt, asking Claude to output only valid JSON.

The system processes product reviews using tool use with a defined schema: rating (integer 1-5), pros (string array), cons (string array), and overall_sentiment (enum: positive, negative, mixed). Testing reveals two issues with brief or ambiguous reviews (~20% of the dataset): (1) for reviews like "Great product!", Claude fabricates specific pros and cons rather than indicating this information isn't explicitly stated, and (2) for sarcastic reviews like "Well that was... interesting", Claude picks sentiment arbitrarily since there's no option for ambiguous cases. What schema modification best addresses both issues?

- ☐ **A** Allow null values for pros/cons, and add "unclear" to the sentiment enum.
- ☒ **B** Allow empty arrays for pros/cons as valid output, and add "unclear" to the sentiment enum.
- ☐ **C** Add an extraction_confidence field (0.0-1.0) for each value, and filter outputs where any confidence falls below a threshold.
- ☐ **D** Make pros and cons optional fields, and add "neutral" and "unclear" to the sentiment enum.

Your team is extracting structured data from 50,000 legacy legal contracts under a two-week deadline. Initial testing with 500 sample documents shows 82% pass JSON schema validation on the first attempt, while the remaining 18% fail due to diverse issues—missing required fields, malformed dates, and incorrectly identified parties. Documents that fail typically need 2-3 prompt refinements targeting their specific failure modes before extraction succeeds. Which batch processing strategy is the most cost-efficient while still meeting the deadline?

- ☐ **A** Use the real-time API for all 50,000 documents since the batch API's 24-hour processing window creates unacceptable deadline risk.
- ☐ **B** Submit all 50,000 documents via batch API, then submit failed extractions in successive batches—refining prompts between each batch—until all documents pass validation.
- ☒ **C** Process 2,000 sample documents via real-time API to identify failure patterns and refine prompts, then batch process all 50,000 with the optimized prompts.
- ☐ **D** Split documents into 10 sequential batches of 5,000 each, analyzing results and refining prompts between batches to improve extraction quality progressively.

Monitoring reveals that specifications sometimes appear inconsistently within source documents. For example, a summary section might state "Battery: 4000mAh" while the detailed specs table shows "Battery: 4200mAh." Your current schema uses a single `battery_capacity_mah` field. This inconsistency occurs in approximately 15% of documents, and historical analysis confirms the detailed specs table is more accurate 90% of the time. What's the most effective approach?

- ☐ **A** Implement schema validation that rejects results containing conflicting values, requiring source document correction before processing.
- ☒ **B** Change to an array field that captures all values found with their source locations, letting downstream systems apply precedence rules.
- ☐ **C** Include extraction instructions specifying to prefer values from the detailed specs table when multiple values exist, keeping the single-value schema.
- ☐ **D** Add a `conflict_detected` boolean field that flags inconsistencies, triggering manual review for affected documents.

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- ☐ A Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.
- ☒ B Route standard reports to the Batch API for 50% cost savings, and route urgent exception reports to the real-time Messages API.
- ☐ C Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.
- ☐ D Submit all documents to the Batch API with custom_ids for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by custom_id. What's the most cost-effective approach to process these failures?

- ☐ **A** Increase the max_tokens parameter for the 300 failed documents and resubmit them in a new batch
- ☒ **B** Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions
- ☐ **C** Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts
- ☐ **D** Resubmit the entire 10,000 document batch using a model tier with a larger context window

Your `publish_article` tool calls an external CMS API that occasionally returns transient errors (network timeouts, 503s) and non-transient errors (403 permission denied, 422 validation failure). Currently, every error is returned directly to the agent, which leads to the agent retrying non-transient errors and wasting turns on failures that will never succeed. How should you partition error-handling responsibility between the tool implementation and the agent?

- ☐ **A** Implement a universal error handler that catches all exceptions and returns a generic "tool unavailable—try again later" message, shielding the agent from error complexity.
- ☐ **B** Handle all errors inside the tool: implement retries with exponential backoff for every error type, and only surface a failure to the agent after a fixed number of retry attempts have been exhausted.
- ☐ **C** Surface all errors to the agent immediately with detailed context, and let the agent decide which errors to retry and how many times—keeping the tool implementation stateless and simple.
- ☒ **D** Handle transient errors (timeouts, 503s) with automatic retries inside the tool implementation, and surface non-transient errors (permission denied, validation failures) to the agent with descriptive messages so it can take corrective action.

Your agent has access to 50+ specialized API connectors for different external services. As the connector library grew, tool selection accuracy dropped to 58%. You design a `search_connectors(description)` tool that finds matching connectors, but in testing, agents frequently skip searching and call connectors directly (often incorrectly), or search correctly but still select wrong connectors from the filtered results. How should you design the tool composition pattern to address both issues?

- ☐ **A** Enhance all connector descriptions with detailed usage examples, edge cases, and input requirements. Add few-shot examples showing the correct search-then-use workflow.
- ☐ **B** Design connectors with built-in compatibility validation that returns descriptive errors for mismatched requests.
- ☐ **C** Design a `find_and_execute(description, params)` composite tool that searches and immediately executes the best-matching connector.
- ☒ **D** Design `search_connectors` to dynamically add matched connectors to the agent's available tools. Connectors start unavailable and persist once discovered.

Your `remove_team_member` tool uses a `dry_run: boolean` parameter for previewing impacts before execution. Production monitoring shows the agent bypasses the preview step in 15% of cases by calling with `dry_run=false` directly. You need to ensure every removal is preceded by a preview that the user explicitly confirms. What is the most reliable approach?

- ☒ **A** Replace with two tools: `preview_remove_member` returns impact details and a single-use confirmation token; `execute_remove_member` requires that token, binding execution to the specific previewed action.
- ☐ **B** Annotate the tool as requiring confirmation and configure the orchestration layer to prompt the user for approval before forwarding any calls to annotated tools.
- ☐ **C** Add server-side validation that permits `dry_run=false` only when a `dry_run=true` call with identical parameters occurred within the past 60 seconds.
- ☐ **D** Add detailed instructions and few-shot examples to the tool description requiring the agent to always call with `dry_run=true` first and wait for user confirmation before calling with `dry_run=false`.

Your expense reimbursement agent processes employee requests using a `process_reimbursement` tool. Company policy requires that reimbursements above \$500 must be approved by a manager before funds are disbursed. The agent handles hundreds of requests daily, and you need the threshold enforcement to be tamper-proof regardless of how the agent is prompted. Which design ensures the \$500 approval threshold cannot be bypassed?

- ☐ **A** The `process_reimbursement` tool accepts an `approved_by_manager: boolean` parameter. The system prompt instructs the agent to only set this to `true` after confirming that a manager has approved the request. A nightly audit script reviews all reimbursements where `approved_by_manager` was set to `true`.
- ☐ **B** The `process_reimbursement` tool accepts amount and details, and internally enforces the threshold: amounts $\leq$ \$500 are auto-disbursed and the tool returns a success confirmation; amounts $>$ \$500 cause the tool to create a pending approval request and return a status indicating manager review is pending.
- ☐ **C** Provide two tools: `auto_reimburse` (hard-coded limit of \$500) and `request_manager_approval`. Include detailed system prompt instructions telling the agent to check the amount and call the appropriate tool. Add a `PostToolUse` hook that logs which tool was called for auditing.
- ☒ **D** Implement the threshold check in a `PreToolUse` hook that inspects the amount parameter before `process_reimbursement` executes. If the amount exceeds \$500, the hook modifies the tool call to add a `requires_approval: true` flag, which the tool checks before disbursing.

Your `search_documents(query)` tool returns results as: "Found 3 documents: 'Q1 Budget Proposal', 'Q2 Budget Forecast', 'Annual Review'". You want the agent to chain this with action tools like `share_document(document_id, email)` and `move_document(document_id, folder)`. What return format would best enable these multi-step workflows?

- ☐ **A** A JSON array of document titles extracted from the search results
- ☒ **B** Structured data containing document IDs and metadata for each result
- ☐ **C** More detailed human-readable descriptions including file sizes and authors
- ☐ **D** URLs that users can click to access documents in their browser

Your `update_user_profile` tool accepts a `user_id` (required) and an optional `fields_to_update` object. In testing, Claude frequently omits `user_id` or passes incorrectly structured data. What is most critical for helping Claude understand what parameter values to provide?

- ☐ A Verbose parameter names encoding format hints, such as `user_id_string_uuid_format`
- ☐ B Clear parameter descriptions explaining expected format, such as "`user_id` : UUID of the user to update (required)"
- ☐ C Detailed error responses explaining why invalid parameter values were rejected
- ☒ D Strict JSON Schema type constraints marking `user_id` as required and defining `fields_to_update` as an object type

Your `search_documents` tool needs a parameter to specify which database to search. Your organization has three document databases: "research_papers", "internal_reports", and "technical_specs". Users express this naturally in conversation ("search the research database", "check technical documents"). How should you design the database selection parameter?

- ☒ **A** An enum parameter with values ["research_papers", "internal_reports", "technical_specs"], requiring the model to map natural language to the appropriate value.
- ☐ **B** A freeform string parameter where the backend uses semantic matching to determine which database(s) to search.
- ☐ **C** No explicit parameter—search all three databases by default, then have the model filter results by source.
- ☐ **D** A freeform string parameter with runtime validation that returns an error if the value doesn't match a known database.

Your agent includes an `update_game_score` tool that accepts `game_date` (string), `home_team` (string), and `away_team` (string) parameters. Production logs reveal recurring issues: the agent uses team nicknames instead of official names, applies inconsistent date formats, and selects the wrong game when teams have rematches in the same season. What tool interface change would most effectively prevent these errors?

- ☐ **A** Add detailed examples to the tool description showing the required date format and complete list of official team names.
- ☐ **B** Add enum constraints listing valid team names for both team parameters, and add a regex pattern enforcing ISO 8601 format for the date parameter.
- ☒ **C** Replace the three parameters with a single `game_id` parameter and a separate `search_games` lookup tool that returns matching game IDs.
- ☐ **D** Add a `season` parameter to disambiguate rematches, and add a `confirm_before_update` flag that returns the resolved game details for the agent to verify before the score is committed.

Your shipment tracking tool queries multiple carriers (FedEx, UPS, DHL) that each return status information in different formats—FedEx uses numeric codes, UPS uses descriptive phrases, and DHL returns timestamped event arrays. The agent uses these results to determine delivery status and escalate delayed shipments. How should you structure the tool's return value to best enable the agent's reasoning?

- ☒ **A** Return a normalized schema with `status`, `estimated_delivery`, `delay_reason`, and `requires_action` fields, converting carrier formats internally.
- ☐ **B** Return both a normalized summary and the complete raw carrier response in each tool call.
- ☐ **C** Return raw carrier responses with source metadata, encoding carrier-specific interpretation logic in the system prompt.
- ☐ **D** Design separate tool endpoints for each carrier (`track_fedex`, `track_ups`, `track_dhl`) with carrier-specific response schemas.

Your agent has a `log_workout` tool that accepts `exercise_type` (string), `value` (number), and `measurement` (string). Production monitoring shows the agent frequently passes mismatched combinations—using `measurement: "reps"` for cardio exercises like running, or `measurement: "miles"` for strength exercises like bench press. Your exercises naturally divide into two categories: cardio (measured in time or distance) and strength (measured in reps and sets). 23% of tool calls have invalid combinations. What approach would most effectively reduce these errors?

- ☒ **A** Split into `log_cardio_workout` (with `duration_minutes` or `distance_miles` parameters) and `log_strength_workout` (with `reps` and `sets` parameters).
- ☐ **B** Add explicit examples to the tool description showing valid combinations (e.g., "For running: use minutes or miles. For push-ups: use reps") with constraints for each exercise category.
- ☐ **C** Implement server-side validation returning descriptive errors for invalid combinations, allowing the agent to retry with corrections.
- ☐ **D** Add enum constraints on `measurement` limiting values to "minutes", "miles", "reps", or "sets" to prevent arbitrary measurement strings.

Your scheduling agent uses `get_available_slots(date, provider_id)` to retrieve open appointment times, then `book_appointment(provider_id, slot_time, patient_id)` to reserve a slot. Support tickets show that 15% of booking attempts fail with "slot no longer available" because another user booked the slot between the availability check and the booking call. How should you redesign these tools?

- ☐ **A** Keep both tools but add retry logic to the agent's system prompt, instructing it to call `get_available_slots` again and select a different time if booking fails.
- ☒ **B** Combine both tools into a single `find_and_book_appointment` that atomically checks availability and books, returning either the confirmed booking or available alternatives.
- ☐ **C** Modify `book_appointment` to return detailed failure information including currently available alternative slots when the requested slot is unavailable, enabling the agent to retry with a different time.
- ☐ **D** Add a `hold_slot(provider_id, slot_time)` tool that creates a 60-second temporary reservation, requiring the agent to call it between checking availability and booking.

Your `post_content` tool requires user confirmation before publishing. The current workflow displays "Ready to post to social media. Confirm?" and analytics show users approve 98% of requests within 2 seconds. Post-mortems reveal incidents where posts went to wrong accounts, were scheduled for wrong times, or contained errors—all confirmed by users without catching the mistakes. How should you redesign the confirmation workflow?

- ☐ **A** Require users to type a confirmation phrase instead of clicking a button
- ☐ **B** Add a mandatory waiting period before the confirm option becomes available
- ☐ **C** Auto-approve routine posts and only require explicit confirmation for unusual patterns like posting to new accounts or large audiences
- ☒ **D** Include the complete post text, target account, scheduled time, and platform in the confirmation request

Your CRM agent's `delete_contact` tool handles requests like "delete the duplicate entry for Acme Corp." The database contains similarly named records (e.g., "Acme Corp," "Acme Corporation," "ACME Corp Inc."), and analytics show 8% of deletions are reversed within 24 hours due to misidentified records. Users have also complained that the current multi-step confirmation flow adds too much friction to routine cleanup tasks. Which approach most effectively reduces the error rate while maintaining workflow efficiency?

- ☐ A Implement soft-delete with a 30-day recovery window so users can undo mistakes without slowing down the deletion workflow.
- ☒ B Present matched records with differentiating fields and require single-click confirmation of the intended target before executing deletion.
- ☐ C Deploy automated duplicate detection that identifies and merges probable duplicates, removing the need for manual deletion requests.
- ☐ D Require users to supply the exact record ID from the CRM interface rather than using natural language references to contact names.

Your product search tool queries an external catalog API and returns matching items. In production, you observe the agent frequently retries searches immediately after receiving zero results, treating "no matches found" as a failure requiring retry. The external API returns HTTP 200 with an empty results array—a valid response. How should you restructure the tool's result to help the agent correctly interpret empty result sets?

- ☐ **A** Return a natural language string describing the outcome, allowing the agent to interpret the result contextually based on the message content.
- ☐ **B** Add a `suggestions` field containing alternative search strategies when results are empty, helping guide the agent toward more productive follow-up queries.
- ☒ **C** Return a structured result with a `success` boolean and `results` array, reserving `isError: true` for actual execution failures only.
- ☐ **D** Return a result object with `isError: true` and a message explaining no products matched.

Your agent uses three tools: `get_property_details(property_id)` returns data including street address, `get_price_history(property_id)` returns historical pricing, and `get_neighborhood_info(address)` returns area statistics. You observe that `get_neighborhood_info` always requires calling `get_property_details` first just to extract the address, even when users specify the property by ID. This creates unnecessary latency and failure coupling—if the first call fails, the neighborhood request also fails. What tool design change best addresses this?

- ☐ **A** Add retry logic and timeout handling to `get_property_details`.
- ☒ **B** Change `get_neighborhood_info` to accept `property_id`, resolving the address internally.
- ☐ **C** Create a `lookup_address(property_id)` helper tool for retrieving addresses.
- ☐ **D** Consolidate into a single `get_property_with_neighborhood(property_id)` tool returning both datasets.

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services. What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- ☐ **A** Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.
- ☐ **B** Read all files containing "auth", "login", "permission", or "token" in their content or filename.
- ☒ **C** Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.
- ☐ **D** Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.

An engineer asks your agent to identify untested code paths in a legacy payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate—it's forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows. What's the most effective approach to complete this investigation?

- ☐ **A** Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.
- ☒ **B** Spawn subagents to investigate specific questions (e.g., "find all test files for payment processing," "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level understanding.
- ☐ **C** Switch to using Grep to search for specific function names instead of reading full files, reducing the content loaded into context for remaining exploration.
- ☐ **D** Clear context with `/clear`, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

An engineer sees an unfamiliar error message "SYNC_CONFLICT: entity version mismatch detected" in production logs but doesn't know which of the 12 services in the codebase generates it. They ask the agent to help locate the source code. What exploration approach will most efficiently find the responsible code?

- ☐ **A** Read the project's README and service configuration files to understand the architecture, then systematically Read source files in each service directory.
- ☐ **B** Use Grep to find all files that import the project's error handling module, then Read those files to locate custom error definitions.
- ☐ **C** Use Glob to find files in directories commonly associated with error handling (such as `errors/`, `exceptions/`, or `handlers/`) across all services, then Read each matching file.
- ☒ **D** Use Grep to search for distinctive text from the error message (like "SYNC_CONFLICT" or "entity version mismatch"), then Read the matching files to understand context.

An engineer submits two requests:

- Request A: "Rename the `getUserData` function to `fetchUserProfile` everywhere it's used."
- Request B: "Improve error handling throughout the data processing module—add try/catch blocks, meaningful error messages, and ensure failures don't silently corrupt data."

For which request does specifying an explicit multi-phase workflow (such as analyze → propose → implement with review) most improve outcome quality?

- ☐ **A** Both requests benefit equally
- ☒ **B** Request B, the error handling task
- ☐ **C** Request A, the function rename task
- ☐ **D** Neither request benefits significantly

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge. What approach best balances efficiency and accuracy?

- ☐ **A** Resume the session and immediately have the agent re-read all 12 previously analyzed files
- ☐ **B** Resume the session without informing the agent about the changed files
- ☒ **C** Resume the session and inform the agent which specific files changed for targeted re-analysis
- ☐ **D** Start a fresh session to ensure the agent works with current codebase state without stale assumptions

After adding an MCP server with specialized code refactoring tools (`extract_function` , `rename_variable` , `inline_function`), you notice the agent still uses basic text manipulation via Write and Bash sed commands for refactoring tasks. The MCP server is connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like "extract_function: Extracts a function from code."

What's the most effective way to improve adoption of the MCP refactoring tools?

- ☐ **A** Remove the Write tool from the agent's configuration for refactoring sessions so it must use the MCP tools for code modifications.
- ☐ **B** Accept this as expected behavior since simpler tools like sed are more predictable than specialized refactoring tools.
- ☒ **C** Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.
- ☐ **D** Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.

An engineer asks the agent to find all callers of a function before removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module). What exploration strategy will most reliably identify all callers?

- ☐ **A** Use Grep to search for the function's original name across the codebase.
- ☐ **B** Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.
- ☒ **C** Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.
- ☐ **D** Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The Edit tool fails because its `old_string` parameter cannot find unique text to match — the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?

- ☐ **A** Use Bash to append the function definition to the end of the file using heredoc syntax
- ☐ **B** Use Edit with an extremely long `old_string` capturing 30+ lines of context to guarantee uniqueness
- ☒ **C** Use Read to load the file, add the function at the appropriate location, then Write the updated file
- ☐ **D** Use Edit's `replace_all` parameter to target a common pattern and embed the new function in the replacement text

You're building a security scanning workflow. When engineers need to locate all occurrences of a dangerous function like `eval()` across a large codebase, which tool should your agent use for content search?

- ☐ **A** Read the project's main entry file and follow import statements to trace where `eval` might be used.
- ☐ **B** Use Glob with a pattern like `**/*eval*` to find files, then Read each matching file.
- ☐ **C** Use Bash to run `ls -R | grep eval` to recursively list files containing `eval`.
- ☒ **D** Use Grep to search for the pattern `"eval("` across all files in the codebase.

Your agent has analyzed a complex service module—reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

- ☐ **A** Start two fresh sessions, having each re-read the relevant source files before beginning.
- ☐ **B** Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.
- ☒ **C** Resume the analysis session with `fork_session` enabled, creating a separate branch for each testing strategy.
- ☐ **D** Export the analysis session's key findings to a file, then create two new sessions that reference this file.

After integrating a local MCP server providing code analysis tools (`analyze_dependencies` , `find_dead_code` , `calculate_complexity`), you verify the server is healthy and tools appear in the `tools/list` response. However, you observe that the agent consistently uses Grep to search for import statements instead of calling `analyze_dependencies` —even when users explicitly ask about "code dependencies."

Examining tool definitions reveals:

- MCP: `analyze_dependencies` — "Analyzes dependency graph"
- Built-in: Grep — "Search file contents for a pattern using regular expressions. Returns matching lines with line numbers and surrounding context."

What's the most effective approach to improve the agent's selection of MCP tools?

☐ **A** Split `analyze_dependencies` into granular tools (`list_imports` , `resolve_transitive_deps` , `detect_circular_deps`) so each has a focused purpose less likely to overlap with Grep.

☐ **B** Add routing instructions to the system prompt specifying that dependency-related questions should use MCP tools rather than Grep.

☒ **C** Expand MCP tool descriptions to detail capabilities and outputs—e.g., "Builds dependency graph showing direct imports, transitive dependencies, and cycles."

☐ **D** Remove Grep from available tools when the MCP server is connected to eliminate functional overlap.

A developer asks the agent to investigate why a specific API endpoint intermittently returns 500 errors. The codebase has 200+ files and the developer doesn't know which components are involved. The agent must trace the error through routing, middleware, business logic, and database layers. What task decomposition approach would be most effective?

- ☐ **A** Define a fixed sequence of investigation steps upfront—grep for error patterns, then read error handlers, then check database queries, then examine middleware—executing each step regardless of intermediate findings.
- ☐ **B** Run parallel worker agents that simultaneously investigate all four layers, then synthesize their findings to identify where the error originates.
- ☐ **C** Have the agent first create a comprehensive plan mapping all code paths through the endpoint before beginning any file exploration or code reading.
- ☒ **D** Have the agent dynamically generate investigation subtasks based on what it discovers at each step, adapting its exploration plan as new information about the error path emerges.

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth—having the agent propose specific code changes for each—before deciding which to implement. What's the most effective way to structure this exploration?

- ☒ **A** Use `fork_session` to create two branches from yesterday's analysis, exploring one approach in each fork.
- ☐ **B** Resume yesterday's session and explore both approaches sequentially within the same conversation thread.
- ☐ **C** Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.
- ☐ **D** Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

An engineer used Claude Code yesterday to investigate authentication flows in a legacy monolith, building up significant context over a 2-hour session. Today she wants to continue that specific investigation. She's worked on three other codebases since then and knows the session was named "auth-deep-dive". How should she resume?

☐ **A** Use `--continue` to pick up where the most recent conversation left off

☒ **B** Use `--resume auth-deep-dive` to load that specific session by name

☐ **C** Use `--session-id` with the UUID from yesterday's session transcript file

☐ **D** Start fresh and re-read the same files

An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's the most effective next step for building understanding while managing context constraints?

- ☐ **A** Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.
- ☐ **B** Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.
- ☒ **C** Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.
- ☐ **D** Use Glob to find files matching common caching patterns (cache.py, caching/), prioritize the largest files by reading them first, then check smaller files for gaps.

Users frequently refine their search criteria mid-conversation. You notice a pattern: when users say things like "Actually, let's raise the budget to \$650K" or "I'd prefer a condo now instead of a house," the assistant sometimes continues referencing the original preferences in later responses—even though the updates are clearly present in the conversation history. Context usage is only at 35% capacity. Which solution most reliably ensures the model uses the current preferences?

- ☒ **A** Maintain a structured state object with current preferences, update it on changes, and include it in each request.
- ☐ **B** Implement conversation pruning to remove turns containing outdated preferences, ensuring only current ones remain in context.
- ☐ **C** Include few-shot examples showing the assistant correctly acknowledging and applying preference changes in responses.
- ☐ **D** Add system prompt instructions emphasizing that the model should always prioritize the most recently stated preferences over earlier ones.

Your conversational AI tutor has a 2,800-token system prompt containing teaching methodology, persona guidelines, and detailed written instructions for adapting explanations to different proficiency levels. User testing reveals that in conversations exceeding 12 turns (approximately 4,000 tokens of conversation history), the assistant increasingly ignores the proficiency-adaptation guidelines, defaulting to intermediate-level explanations regardless of the learner's stated level. What's the most effective approach to ensure consistent adherence to these guidelines throughout extended conversations?

- ☐ **A** Inject a condensed reminder of the proficiency requirements into the conversation as a system message every 4-5 turns.
- ☐ **B** Restructure the system prompt to place the proficiency-adaptation rules in a clearly-marked final section immediately before the conversation history begins.
- ☐ **C** After each assistant response, make a separate API call to evaluate whether the difficulty level matched the learner's profile, regenerating responses that don't align.
- ☒ **D** Replace the verbose proficiency guidelines with few-shot examples demonstrating appropriate responses at each proficiency level, showing concrete differences in vocabulary, complexity, and explanation depth.

After 30+ turns, your conversational assistant shows noticeably slower responses and occasionally produces less coherent outputs. Investigation reveals: (1) average conversations reach 50,000 tokens by turn 35, (2) production logs show 94% of user messages only reference the previous 3-5 exchanges, (3) the 6% of queries referencing earlier context typically ask about information the user could easily re-state. Your goal is to maintain response speed and quality while maintaining good user experience. What's the most effective approach?

- ☐ **A** Build a retrieval system that stores all conversation turns and uses semantic search to pull in relevant historical context only when the current query appears to reference past information.
- ☒ **B** Implement a summarization layer that progressively compresses older conversation turns into a running summary while keeping the most recent 5-6 turns verbatim, maintaining full historical context in condensed form.
- ☐ **C** Implement a sliding window keeping only the system prompt and last 8-10 turns. When users reference earlier context, acknowledge the limitation and ask them to re-state the relevant information.
- ☐ **D** Enable prompt caching and continue sending the complete conversation history, using cached prefixes to reduce per-request costs while preserving all context.

During QA testing, you notice that Claude follows your system prompt guidelines consistently in the first 10-15 turns, but by turn 25-30, responses begin deviating—using informal tone when formality was specified, occasionally skipping required formatting, or providing information types the guidelines restrict. Conversation length is well within context limits (typically 30,000 tokens out of 200,000 available). What's the most effective approach to maintain consistent behavior throughout extended conversations?

- ☐ **A** Move behavioral guidelines from the system prompt into the first user message.
- ☐ **B** Automatically start a new conversation after 20 turns, passing a summary of the prior context to maintain continuity.
- ☐ **C** Insert user-role messages that reinforce critical guidelines at natural conversation breakpoints, especially before complex requests.
- ☒ **D** Implement post-response validation that regenerates each response until it conforms to the specified guidelines.

Performance analysis reveals your context is composed of accumulated RAG results from all previous queries, which is crowding out conversation history and causing coherence degradation after 15+ turns. Which approach best addresses this issue?

- ☐ **A** Implement a sliding window for RAG results from the last 2-3 queries while preserving conversation history
- ☐ **B** Implement semantic deduplication to identify and remove redundant information across the accumulated RAG results and conversation turns
- ☐ **C** Shift context budget to favor RAG results while reducing conversation history allocation
- ☒ **D** Compress all RAG results into a consolidated summary document that updates incrementally after each retrieval

Your recipe assistant has a system prompt with multiple behavioral requirements: a friendly persona, measurement unit preferences, dietary restriction awareness, and instructions to ask clarifying questions before suggesting ingredient substitutions. During testing, you notice the model consistently makes substitutions without asking clarifying questions first, while following the other instructions correctly. Which system prompt modification is most likely to fix the substitution clarification compliance issue?

- ☒ **A** Organize the prompt into XML-style sections with a substitution block containing behavioral examples.
- ☐ **B** Shorten the system prompt by condensing other sections to increase substitution instruction prominence.
- ☐ **C** Move the substitution clarification instructions to the end of the system prompt for recency emphasis.
- ☐ **D** Add "CRITICAL:" prefixes to the substitution instructions to increase their salience.

You're implementing a feature where users refine their playlist preferences through multiple conversation turns. After deploying, you notice Claude's responses don't reflect what users said earlier in the same conversation—for example, a user says they love jazz, but two messages later Claude asks what genres they enjoy. What is the most likely cause?

- ☐ **A** The model's context window has been exceeded by the conversation length
- ☐ **B** The Claude API requires a `session_id` parameter that you haven't configured
- ☒ **C** Your application isn't including prior messages in the messages array
- ☐ **D** Claude requires a vector database connection to maintain conversation memory

Over several turns discussing investment strategy, a user has stated: "I have a very low risk tolerance—I can't handle seeing my portfolio drop" and later, "I want to maximize my returns—I've seen friends do well in crypto and tech stocks." They now ask "What should I invest in?" Which approach best ensures the resulting recommendation aligns with the user's actual priority?

- ☐ **A** Provide separate recommendations for both scenarios (conservative and growth-oriented) without clarifying which the user prefers, and let them choose.
- ☐ **B** Recommend a balanced portfolio that moderates both preferences without explicitly addressing the conflict.
- ☐ **C** Proceed with the most recently stated preference.
- ☒ **D** Surface the contradiction and ask the user to clarify which matters more—capital preservation or growth potential—before recommending.

During a conversation about order tracking, your external system receives a webhook indicating the user's package has shipped. The user is actively chatting and will likely send a follow-up message soon. You want the assistant to naturally incorporate this status change in its next response. What's the most effective approach?

- ☒ **A** Add the current shipping status to the system prompt before the next API call.
- ☐ **B** Configure the assistant to call a `get_order_status` tool at the start of every response.
- ☐ **C** Immediately send an API request with the update as a synthetic user message, generating an unsolicited assistant response.
- ☐ **D** Append the status update as a prefix to the next user message before calling the API.

After deploying an updated system prompt that improves response quality, users with multi-session conversations spanning several weeks report that the assistant now contradicts its earlier statements and has a noticeably different communication style. New users don't experience these issues.

What's the best approach to resolve this?

- ☐ **A** Add instructions to the new system prompt directing the assistant to maintain consistency with any prior statements in the conversation history.
- ☒ **B** Version system prompts and associate each conversation with the prompt version under which it started, applying updates only to new conversations.
- ☐ **C** Add a transition message when sessions resume explaining that the assistant has been updated and behavior may differ.
- ☐ **D** Regenerate summaries of existing conversations using the new prompt and replace the stored histories to align past context with current behavior.

Your fitness coaching assistant uses a system prompt with detailed conditional logic: "If the user mentions being a beginner, provide step-by-step form instructions. If they use terms like 'progressive overload' or 'superset', respond concisely. If they ask about injury history, always recommend consulting a physician." During evaluation, you find the assistant correctly adapts to explicit expertise declarations but struggles when users don't clearly state their level—often defaulting to overly detailed responses regardless of contextual cues like technical terminology. Which change to the system prompt would most directly address this failure to pick up on implicit expertise signals?

- ☐ **A** Add an explicit instruction for the model to ask a clarifying question about experience level whenever the user's expertise isn't immediately clear from their first message.
- ☐ **B** Implement a pre-conversation intake that asks users to rate their experience level, then inject that rating into the system prompt as context for all subsequent responses.
- ☐ **C** Add more conditional branches to cover additional expertise signals, such as "If user mentions specific rep ranges or asks about periodization, treat as advanced."
- ☒ **D** Replace most conditionals with a general principle: "Adapt explanation depth to match user expertise, mirroring their terminology." Keep only the safety-critical conditional about injury consultations.

Users frequently send ambiguous requests like "book a venue for the party" without specifying date, guest count, or budget. Your evaluation shows the assistant asks an average of 4.2 clarifying questions before taking action, causing 35% of users to abandon mid-conversation. However, when you reduce questions, users sometimes receive recommendations that don't match their preferences. What's the most effective approach to improve this trade-off?

- ☐ **A** Configure the assistant to consolidate all clarifying questions into a single compound question (e.g., "What date, guest count, and budget are you considering?") to reduce the total number of conversational turns.
- ☐ **B** Configure the assistant to proceed with reasonable defaults (medium-sized venue, next weekend, moderate budget) without explicitly stating these assumptions, allowing users to provide corrections if results don't match expectations.
- ☒ **C** Instruct the assistant to state explicit assumptions based on conversation history, proceed with recommendations while inviting corrections, and reserve clarifying questions only for irreversible actions like confirming bookings.
- ☐ **D** Implement a structured intake form that collects all required parameters (date, guest count, budget, venue type) upfront before the assistant begins providing any recommendations.

After a 40-minute session helping plan a dinner party, the conversation has grown to 78,000 tokens. The history includes: (1) the user mentioning a guest has a severe shellfish allergy, (2) specific measurements for scaling recipes to 8 servings, (3) the user's clarification that "room temperature butter" means 68°F in their kitchen, and (4) general back-and-forth about meal timing and presentation. You need to implement context management before the window limit is reached. What approach best balances information preservation with token reduction?

- ☒ **A** Extract critical structured data (allergies, serving counts, user-defined terms) into a compact reference section, summarize general discussion, and retain recent exchanges verbatim.
- ☐ **B** Summarize the entire conversation history into a concise summary capturing main topics discussed, then append new messages going forward.
- ☐ **C** Implement a sliding window retaining only the most recent 20,000 tokens, relying on users to re-state important information when relevant.
- ☐ **D** Store the full conversation externally and use semantic search to retrieve relevant portions for each turn, loading only matching segments into context.

Your music discovery assistant should consistently maintain an enthusiastic tone, explain its reasoning for each recommendation, and ask clarifying questions to better understand user preferences. You want this behavior to persist reliably across all user interactions. Where should you define these behavioral guidelines?

- ☐ **A** Prepend to each user message before sending to the API
- ☐ **B** In environmental variables that your application passes to the API client
- ☒ **C** In the system prompt
- ☐ **D** In the first assistant message, instructing Claude to follow these guidelines going forward

Your conversation history includes two types of content: persistent story elements (character backgrounds, plot structure, world rules) that must remain consistent throughout, and extensive brainstorming discussion that's mostly ephemeral. After 40+ turns, you're hitting context limits and users report the assistant "forgets" established character traits, breaking narrative consistency.

Which approach best ensures persistent story elements remain available to the model while reclaiming context space?

- ☐ **A** Apply a sliding-window approach keeping only the most recent 25 turns, relying on the model to infer earlier context from recent discussion flow.
- ☐ **B** Summarize the entire conversation history into a condensed synopsis every 20 turns, replacing the full history to free up tokens.
- ☒ **C** Separate persistent story elements into a retained "story bible" section at context start, applying trimming or summarization only to brainstorming discussion.
- ☐ **D** Store all history in a vector database and retrieve semantically similar passages for each new message, replacing conversation history with retrieved chunks.